



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 03/10

Backend FastAPI Complet

Python + FastAPI + Async + Pydantic + SQLAlchemy

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - C'est quoi un backend ?	3
Chapitre 2 - C'est quoi FastAPI ?	6
Chapitre 3 - Async/await en Python	10
Chapitre 4 - Anatomie d'une route	14
Chapitre 5 - Pydantic et validation	18
Chapitre 6 - Dependency Injection	22
Chapitre 7 - Routers et organisation	25
Chapitre 8 - Lifespan et configuration	29
Chapitre 9 - chatbot-service decortique	32
Chapitre 10 - academic-service decortique	38
Chapitre 11 - Gestion des erreurs	43
Chapitre 12 - Swagger UI automatique	46
Chapitre 13 - Conclusion	49

Chapitre 1 - C'est quoi un backend ?

Le **backend** est la partie serveur d'une application : ce qui tourne dans le cloud (ou sur ton PC en mode dev) et que l'utilisateur ne voit jamais directement.

■ **ANALOGIE** : Le backend, c'est le **cerveau cache du systeme**. Tu vois la bouche (frontend) parler, mais c'est le cerveau qui pense. Si tu coupes le backend, le frontend reste affiche mais ne peut plus rien faire d'utile.

1.1 Roles du backend

- **Recevoir** les requetes HTTP du frontend
- **Authentifier** l'utilisateur (qui es-tu ?)
- **Autoriser** (as-tu le droit ?)
- **Valider** les donnees recues (sont-elles correctes ?)
- **Calculer** la reponse (logique metier)
- **Interroger** les bases de donnees
- **Serializer** la reponse en JSON
- **Logger** ce qui s'est passe (debug, monitoring)

1.2 Backend monolithe vs micro-services

Critere	Monolithe	Micro-services
Architecture	1 grosse app	N petites apps
Deploiement	Tout d'un coup	Service par service
Resilience	Si crash = tout down	Crash isole
Complexite	Plus simple	Plus dur (reseau)
Performance	Bon (pas de reseau)	Latence inter-services
Equipes	1 grosse equipe	N petites equipes
FSBM	Non choisi	6 services planifies

1.3 Choix du langage

Pour le backend, on aurait pu choisir :

- **Python** : choisi - syntaxe simple, ecosysteme ML/IA riche (sklearn, NLTK)
- **Node.js** : tres rapide, mais asynchronisme verbeux pour debutant
- **Go** : performant, mais courbe d'apprentissage

- Java : robuste, mais lourd pour un PFE
- PHP : dat█©, peu adapte aux micro-services modernes
- Rust : ultra-rapide, mais experts seulement

[★] Python a ete choisi pour 3 raisons : (1) syntaxe accessible aux debutants, (2) librairies NLP/ML deja en place (sklearn, NLTK, transformers), (3) FastAPI moderne et performant.

Chapitre 2 - C'est quoi FastAPI ?

2.1 Naissance et popularite

FastAPI est un framework Python cree en 2018 par Sebastian Ramirez (alias @tiangolo). Il a explose en popularite grace a ses 3 atouts uniques : performance + validation auto + documentation auto.

2.2 Hello World FastAPI

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/')
async def hello():
    return {'message': 'Hello World !'}

# Lancer :
# py -m uvicorn main:app --reload
# -> http://localhost:8000
# -> http://localhost:8000/docs (Swagger UI auto !)
```

2.3 Trois forces uniques

Force 1 : Performance

FastAPI est base sur Starlette (ASGI async) et Pydantic. Il est aussi rapide que Node.js et Go pour la plupart des cas. C'est l'un des frameworks Python les plus rapides au monde.

Force 2 : Validation automatique avec Pydantic

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    message: str = Field(..., min_length=1, max_length=500)
    session_id: str | None = None

@app.post('/chat')
async def chat(req: ChatRequest):
    # Si l'utilisateur envoie un message vide ou > 500 chars,
    # FastAPI rejette automatiquement avec 422
    return {'received': req.message}
```

Force 3 : Documentation Swagger automatique

FastAPI genere automatiquement une interface interactive Swagger UI sur /docs et ReDoc sur /redoc. Le jury peut tester les endpoints en direct !

[■] L'URL <http://localhost:8001/docs> est l'un des grands atouts a montrer en soutenance. Le jury voit instantanement TOUS les endpoints, leurs parametres, leurs types de retour, et peut les tester en cliquant.

2.4 FastAPI vs Flask vs Django

Critere	FastAPI	Flask	Django
Annee	2018	2010	2005
Async natif	OUI	NON	Partiel
Validation	Pydantic native	Manuel	Forms
Doc auto	OUI	Manuel	Manuel
Use case	APIs modernes	Apps simples	Apps complets
Performance	Tres haute	Moyenne	Moyenne
Communaute	Croissante	Enorme	Enorme

Chapitre 3 - Async/await en Python

3.1 Le probleme du I/O bloquant

Quand ton serveur fait un appel a une base de donnees ou a une API externe, l'attente peut prendre 100ms a 2 secondes. Pendant ce temps, en synchrone, le serveur ne peut rien faire d'autre.

```
# Synchrone (bloquant)
def get_data():
    response = requests.get('https://api.example.com') # attend 1 seconde
    return response.json() # 1 seconde de blocage total

# Pendant cette seconde, le serveur ne fait RIEN d'autre.
# Si 100 utilisateurs arrivent en meme temps, ils attendent en file.
```

3.2 La solution async

```
# Asynchrone (non-bloquant)
async def get_data():
    response = await httpx.get('https://api.example.com')
    return response.json()

# Pendant qu'on attend la reponse externe (await),
# Python execute D'AUTRES taches concurrentes.
```

■ **ANALOGIE** : Imagine un serveur de restaurant. **Synchrone** : il prend la commande du table 1, va en cuisine, attend, ramene, puis seulement apres s'occupe de la table 2. **Async** : il prend la commande de la table 1, donne au cuisinier, va prendre la commande de la table 2, etc. Il revient quand un plat est pret.

3.3 Les 3 mots-cles

Mot-cle	Role
async	Declare une fonction comme asynchrone
await	Attend le resultat sans bloquer
asyncio	Bibliotheque standard pour gerer l'event loop

3.4 Exemple dans notre projet

```
@router.get('/api/professors')
async def list_professors(
    department_id: int | None = None,
    page: int = 1,
    page_size: int = 20,
    db: AsyncSession = Depends(get_db),
):
    # Construction de la requete
    stmt = select(Professor)
    if department_id:
        stmt = stmt.where(Professor.department_id == department_id)
```

```
# await : attend le resultat sans bloquer
total = await db.scalar(select(func.count(Professor.id)))

# await : execute la requete asynchrone
result = await db.execute(stmt.limit(page_size))
profs = result.scalars().all()

return PaginatedResponse(items=profs, total=total)
```

3.5 ASGI vs WSGI

WSGI (1999) est l'ancien protocole synchrone Python pour servir HTTP (Flask, Django). ASGI (2018) est l'evolution asynchrone (FastAPI, Starlette, Django 3+).

On utilise **uvicorn** comme serveur ASGI pour heberger FastAPI :

```
py -m uvicorn app.main:app --reload --port 8001
```


Chapitre 4 - Anatomie d'une route

4.1 Le decorateur

```
@app.get('/api/health')          # methode HTTP + URL
@app.post('/api/chat')
@app.put('/api/users/{id}')      # parametre d'URL
@app.delete('/api/items/{id}')
@app.patch('/api/users/{id}')
```

4.2 Parametres de chemin (path)

```
@app.get('/api/filieres/{code}')
async def get_filiere(code: str):
    # code est extrait de l'URL : /api/filieres/SMI -> code = 'SMI'
    return await find_by_code(code)

@app.get('/api/students/{id}')
async def get_student(id: int): # type int -> conversion auto
    return await find_by_id(id)
# GET /api/students/abc -> 422 (pas un int)
```

4.3 Parametres de query string

```
@app.get('/api/filieres')
async def list_filieres(
    type: str | None = None,          # ?type=MASTER
    department_id: int | None = None, # ?department_id=1
    search: str | None = None,        # ?search=info
    is_active: bool = True,           # ?is_active=true
):
    # Tous ces params viennent de l'URL :
    # GET /api/filieres?type=MASTER&search=info&is_active=true
    ...
```

4.4 Body JSON (POST/PUT)

```
class ChatRequest(BaseModel):
    message: str = Field(..., min_length=1, max_length=500)
    session_id: str | None = None
    temperature: float = 0.7

@app.post('/api/chat')
async def chat(req: ChatRequest):
    # FastAPI desinitialise auto le body JSON en ChatRequest
    # Validation auto : si message > 500 chars, 422
    return await process_chat(req.message)
```

4.5 Headers

```
from fastapi import Header

@app.get('/api/me')
async def get_me(
```

```
authorization: str | None = Header(None), # X-Authorization header
accept_language: str = Header('fr'),
):
    if not authorization:
        raise HTTPException(401, 'Non authentifie')
    user = decode_jwt(authorization.replace('Bearer ', ''))
    return user
```

4.6 Reponse typee

```
class FiliereOut(BaseModel):
    id: int
    code: str
    name: str
    type: str

@app.get('/api/filieres/{code}', response_model=FiliereOut)
async def get_filiere(code: str):
    f = await db.find_by_code(code)
    if not f:
        raise HTTPException(404, 'Filiere introuvable')
    return f # serialise automatiquement en JSON
```

Chapitre 5 - Pydantic et validation

5.1 Le probleme de la validation

Sans validation, ton serveur peut recevoir n'importe quoi (string au lieu d'int, champs manquants, chaines trop longues...). Tu dois ecrire des kilometres de checks.

5.2 Pydantic en 1 minute

```
from pydantic import BaseModel, Field, EmailStr

class StudentCreate(BaseModel):
    cne: str = Field(..., min_length=10, max_length=10)
    first_name: str = Field(..., min_length=2, max_length=80)
    last_name: str = Field(..., min_length=2, max_length=80)
    email: EmailStr # validation email auto
    age: int = Field(..., ge=17, le=80) # entre 17 et 80
    gender: str = Field(..., pattern='^(M|F)$') # regex
```

5.3 Validation automatique

Quand FastAPI recoit cette requete :

```
POST /api/students
{
  "cne": "123",           # trop court !
  "first_name": "K",      # trop court !
  "email": "pas-un-email", # invalide !
  "age": 200,             # hors range !
  "gender": "X"           # mauvais pattern !
}
```

FastAPI repond automatiquement avec 422 et un detail des erreurs :

```
{
  "detail": [
    {
      "loc": ["body", "cne"],
      "msg": "String should have at least 10 characters",
      "type": "string_too_short"
    },
    {
      "loc": ["body", "email"],
      "msg": "value is not a valid email address",
      "type": "value_error.email"
    }
  ]
}
```

Zero ligne de code de validation dans notre route ! Tout est dans le schema Pydantic.

5.4 Validateurs custom

```
from pydantic import field_validator
```

```
class ChatRequest(BaseModel):
    message: str

    @field_validator('message')
    @classmethod
    def no_spam(cls, v: str) -> str:
        if 'spam' in v.lower():
            raise ValueError('Le mot spam est interdit')
        return v.strip() # transforme et retourne
```

5.5 Settings (Pydantic Settings)

Pour la configuration depuis le fichier .env, on utilise pydantic-settings :

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    service_name: str = 'chatbot-service'
    service_port: int = 8001
    db_host: str = 'localhost'
    db_password: str = ''
    groq_api_key: str = ''

    model_config = SettingsConfigDict(
        env_file='.env',
        env_file_encoding='utf-8',
        case_sensitive=False,
        extra='ignore',
    )

settings = Settings() # charge .env automatiquement
print(settings.db_password) # 'monpassword' lu depuis .env
```

Chapitre 6 - Dependency Injection

6.1 Le concept

DI = Dependency Injection. Au lieu que chaque route cree ses propres dependances (connexion DB, auth, etc.), FastAPI les **injecte** automatiquement.

■ **ANALOGIE** : Au lieu que chaque cuisinier fabrique sa propre poele (chronophage et duplique), il y a un magasinier qui distribue les poeles. Le cuisinier dit 'j'ai besoin d'une poele' et la recoit automatiquement.

6.2 Depends() en action

```
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession

# Definition de la dependance
async def get_db() -> AsyncSession:
    async with SessionLocal() as session:
        yield session
    # apres le yield : cleanup automatique a la fin de la requete

# Utilisation dans une route
@router.get('/api/filieres')
async def list_filieres(db: AsyncSession = Depends(get_db)):
    # FastAPI appelle get_db(), recupere la session, la passe en parametre
    result = await db.execute(select(Filiere))
    return result.scalars().all()
    # Apres la requete : get_db() reprend la main pour fermer la session
```

6.3 Avantages

- + **Reusabilite** : la meme dependance dans 50 routes
- + **Testabilite** : on peut remplacer get_db par un mock
- + **Composition** : une dependance peut utiliser une autre
- + **Lifecycle automatique** : creation/destruction geres pour toi

6.4 Dependance avec dependances

```
async def get_token(authorization: str = Header(None)):
    if not authorization:
        raise HTTPException(401, 'Token manquant')
    return authorization.replace('Bearer ', '')

async def get_current_user(
    token: str = Depends(get_token),
    db: AsyncSession = Depends(get_db),
):
    user_id = decode_jwt(token)
    user = await db.get(User, user_id)
```

```
    if not user:
        raise HTTPException(401, 'User introuvable')
    return user

@router.get('/api/me')
async def me(user: User = Depends(get_current_user)):
    return user
```

Chapitre 7 - Routers et organisation

7.1 Pourquoi des routers ?

Quand on a 30+ routes, on les regroupe par thematique dans des **routers**. C'est plus organise et modulaire.

7.2 Structure de notre chatbot-service

```
app/  
  main.py          # Bootstrap, include les routers  
  routers/  
    chat.py        # POST /api/chat, /api/chat/feedback  
    llm.py         # POST /api/llm/chat, GET /api/llm/status  
    intents.py     # GET /api/intents  
    system.py      # GET /api/health, /api/stats
```

7.3 Definition d'un router

```
# routers/chat.py  
from fastapi import APIRouter, Depends  
  
router = APIRouter(  
    prefix='/api/chat',    # toutes les routes commencent par /api/chat  
    tags=['chat'],        # groupe dans Swagger UI  
)  
  
@router.post('', response_model=ChatResponse)  
async def chat(req: ChatRequest, db: AsyncSession = Depends(get_db)):  
    ...  
  
@router.post('/feedback')  
async def feedback(req: FeedbackRequest):  
    ...  
  
@router.get('/history/{session_id}')  
async def history(session_id: str):  
    ...
```

7.4 Inclure les routers dans main.py

```
# main.py  
from app.routers.chat import router as chat_router  
from app.routers.llm import router as llm_router  
from app.routers.system import router as system_router  
  
app = FastAPI(title='FSBM Chatbot Service')  
  
app.include_router(chat_router)  
app.include_router(llm_router)  
app.include_router(system_router)
```

7.5 Tags = groupes dans Swagger

Les tags affichent un regroupement visuel sur /docs. Si tu as 30 endpoints, c'est essentiel pour la lisibilite.

Chapitre 8 - Lifespan et configuration

8.1 Le concept de lifespan

Le **lifespan** est une fonction qui s'exécute au démarrage ET à l'arrêt de l'application. C'est là qu'on initialise les ressources coûteuses (NLP, DB pool, configuration).

8.2 Code du lifespan FSBM

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # ■■■■ DEMARRAGE ■■■■
    print('Démarrage du chatbot-service')

    # Charger le classifieur NLP
    classifieur = IntentClassifier()
    if not classifieur.load_model():
        classifieur.train()
    set_classifieur(classifieur)

    # Configurer le LLM Service
    llm_svc = LLMService(classifieur=classifieur, ...)
    set_llm_service(llm_svc)

    print('Service prêt')
    yield # APP TOURNE ICI

    # ■■■■ ARRÊT ■■■■
    print('Arrêt du chatbot-service')
    # Cleanup (fermeture connexions, etc.)

app = FastAPI(lifespan=lifespan)
```

8.3 Configuration via Settings

Le fichier `.env` contient les vraies valeurs (mot de passe, cle API). Pydantic Settings les charge au démarrage :

```
.env :
GROQ_API_KEY=gsk_xxx
DB_PASSWORD=monpassword
DB_NAME=fsbm_db

config.py :
class Settings(BaseSettings):
    groq_api_key: str = ''
    db_password: str = ''
    db_name: str = 'fsbm_db'

    model_config = SettingsConfigDict(env_file='.env')

# Au démarrage :
settings = Settings()
```

```
# settings.groq_api_key = 'gsk_xxx' (lu depuis .env)
```

8.4 CORS Middleware

CORS = Cross-Origin Resource Sharing. Par default, le navigateur bloque les requetes entre domaines differents (security). Le backend doit autoriser explicitement le frontend.

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:4200'], # frontend Angular
    allow_credentials=True,
    allow_methods=['*'], # GET, POST, PUT, DELETE...
    allow_headers=['*'],
)
```

Chapitre 9 - chatbot-service decortique

9.1 Vue d'ensemble

```

chatbot-service/
app/
  main.py                # bootstrap FastAPI + lifespan
  core/
    config.py            # Pydantic Settings
    database.py          # session SQLAlchemy MySQL
    memory.py            # memoire conversationnelle RAM
    persona.py           # detection genre/nom
    web_fetcher.py       # scraping fsbm.ma + Facebook
  nlp/
    preprocessor.py      # tokenisation + stopwords
    language_detector.py # FR/EN/Darija
    classifier.py        # TF-IDF + Cosine multi-langue
  llm/
    groq_client.py       # client Groq (LLaMA 3.3)
    hf_client.py         # client HuggingFace (fallback)
    rag.py               # retrieval + prompt building
    llm_service.py       # orchestrateur avec fallback
  models/
    schemas.py           # Pydantic v2 schemas
  routers/
    chat.py              # /api/chat, /feedback, /history
    llm.py               # /api/llm/chat, /status
    intents.py           # /api/intents
    system.py            # /api/health, /api/stats
  data/
    faq_dataset.json     # 28 intents trilingues
    model.pkl            # modele TF-IDF entraine

```

9.2 Endpoint /api/chat detaille

```

@router.post('/', response_model=ChatResponse)
async def chat(req: ChatRequest, db: AsyncSession = Depends(get_db)):
    # 1. Validation deja faite par Pydantic
    classfier = get_classfier()
    session_id = req.session_id or f'sess_{uuid.uuid4().hex}'

    # 2. Memoire conversationnelle
    ctx = memory.get_or_create(session_id, req.user_id)

    # 3. Detection genre/nom
    g = detect_gender(req.message)
    n = detect_name(req.message)
    if g: ctx.user_context['gender'] = g
    if n: ctx.user_context['name'] = n

    # 4. Prediction NLP
    result = classfier.predict(
        req.message, top_k=5,
        forced_language=req.language
    )

```

```
# 5. Personnalisation
response_text = personalize_response(
    result['response'],
    gender=ctx.user_context.get('gender'),
    name=ctx.user_context.get('name'),
    lang=result['language'],
)

# 6. Persistence MySQL
conv_id = await save_conversation(db, ...)

# 7. Memory enrichie
memory.add_turn(session_id, req.message, response_text, ...)

# 8. Reponse JSON
return ChatResponse(
    response=response_text,
    intent=result['intent'],
    confidence=result['confidence'],
    conversation_id=conv_id,
    ...
)
```

Chapitre 10 - academic-service decortique

10.1 Vue d'ensemble

```
academic-service/
app/
  main.py          # bootstrap FastAPI
  core/
    config.py      # settings .env
  db/
    session.py     # engine + sessionmaker SQLAlchemy async
  models/
    entities.py    # 15 modeles ORM mappes MySQL
  schemas/
    academic.py    # 30+ schemas Pydantic
  routers/
    departments.py # /api/departments
    filieres.py    # /api/filieres + filtres + /code/SMI
    modules.py     # /api/modules
    professors.py  # /api/professors (paginé)
    students.py    # /api/students (paginé)
    schedule.py    # /api/schedule
    exams.py       # /api/exams
    announcements.py # /api/announcements + events + clubs
    system.py      # /health, /overview
```

10.2 Endpoint /api/filieres avec filtres

```
@router.get('/', response_model=list[FiliereOut])
async def list_filieres(
    type: str | None = Query(None),
    department_id: int | None = Query(None),
    is_active: bool = Query(True),
    search: str | None = Query(None),
    db: AsyncSession = Depends(get_db),
):
    # Construction dynamique de la query
    stmt = select(Filiere)
    if type:
        stmt = stmt.where(Filiere.type == type)
    if department_id:
        stmt = stmt.where(Filiere.department_id == department_id)
    if is_active is not None:
        stmt = stmt.where(Filiere.is_active == is_active)
    if search:
        like = f'%{search}%'
        stmt = stmt.where(or_(
            Filiere.name.like(like),
            Filiere.code.like(like)
        ))
    stmt = stmt.order_by(Filiere.type, Filiere.name)

    result = await db.execute(stmt)
    return result.scalars().all()
```

10.3 Endpoint /api/overview (dashboard)

```
@router.get('/overview')
async def overview(db: AsyncSession = Depends(get_db)):
    # 8 requetes count() en parallele (async)
    dept_count = await db.scalar(select(func.count(Department.id))) or 0
    filiere_count = await db.scalar(select(func.count(Filiere.id))) or 0
    module_count = await db.scalar(select(func.count(Module.id))) or 0
    prof_count = await db.scalar(select(func.count(Professor.id))) or 0
    student_count = await db.scalar(select(func.count(Student.id))) or 0
    event_count = await db.scalar(select(func.count(Event.id))) or 0
    ann_count = await db.scalar(select(func.count(Announcement.id))) or 0
    club_count = await db.scalar(select(func.count(Club.id))) or 0

    # Repartition par type
    type_counts = (await db.execute(
        select(Filiere.type, func.count(Filiere.id))
        .group_by(Filiere.type)
    )).all()

    return {
        'departments': dept_count,
        'filieres': filiere_count,
        'modules': module_count,
        'professors': prof_count,
        'students': student_count,
        'events': event_count,
        'announcements': ann_count,
        'clubs': club_count,
        'filieres_by_type': {t: c for t, c in type_counts},
    }
```

Chapitre 11 - Gestion des erreurs

11.1 HTTPException

```
from fastapi import HTTPException

@router.get('/api/filieres/{id}')
async def get_filiere(id: int, db: AsyncSession = Depends(get_db)):
    f = await db.get(Filiere, id)
    if not f:
        raise HTTPException(
            status_code=404,
            detail=f'Filiere {id} non trouvee'
        )
    return f
```

11.2 Codes HTTP a utiliser

Code	Quand l'utiliser
400	Bad Request (mauvais format, params invalides)
401	Unauthorized (pas de token, token expire)
403	Forbidden (token OK mais pas les droits)
404	Not Found (ressource introuvable)
409	Conflict (ex: email deja utilise)
422	Unprocessable Entity (validation echouee - auto par FastAPI)
429	Too Many Requests (rate limit)
500	Internal Server Error (bug serveur, log et retourne)

11.3 Exception handler global

```
from fastapi import Request
from fastapi.responses import JSONResponse

@app.exception_handler(ValueError)
async def value_error_handler(request: Request, exc: ValueError):
    return JSONResponse(
        status_code=400,
        content={'detail': str(exc)}
    )

# Toutes les ValueError du code seront converties en 400 automatiquement
```

11.4 Try/except dans les routes

```
@router.post('/api/chat')
async def chat(req: ChatRequest, db: AsyncSession = Depends(get_db)):
    try:
        result = classifieur.predict(req.message)
```

```
except Exception as exc:
    # Log + remontee
    logger.error(f'NLP error: {exc}')
    raise HTTPException(500, f'Erreur NLP: {exc}')

# Persistance tolerante : si MySQL down, on continue
try:
    conv_id = await save_conversation(db, ...)
except Exception:
    conv_id = 0 # mode degrade

return ChatResponse(..., conversation_id=conv_id)
```


Chapitre 12 - Swagger UI automatique

12.1 C'est quoi Swagger ?

Swagger UI (aussi appele **OpenAPI**) est une **interface web interactive** qui documente automatiquement ton API. Tu peux tester chaque endpoint en cliquant.

12.2 Comment y acceder

Une fois ton service FastAPI lance :

- **<http://localhost:8001/docs>** - Swagger UI (interface interactive)
- **<http://localhost:8001/redoc>** - ReDoc (documentation plus belle, non-interactive)
- **<http://localhost:8001/openapi.json>** - le schema OpenAPI en JSON

12.3 Ce qu'on voit sur /docs

- La liste de TOUS les endpoints groupes par **tag**
- Pour chaque endpoint : methode, URL, description
- Schemas Pydantic des bodies (request + response)
- Bouton **Try it out** pour tester en direct
- Generation automatique de la commande curl

12.4 Ameliorer la doc

```
@router.post(
    '/chat',
    response_model=ChatResponse,
    summary='Envoyer un message au chatbot',
    description='''
    Le chatbot detecte la langue (FR/EN/Darija), classifie l'intent,
    genere une reponse personnalisee. Memoire 10 tours.
    ''',
    tags=['chat'],
)
async def chat(req: ChatRequest):
    ...
```

12.5 Exemples dans le schema

```
class ChatRequest(BaseModel):
    model_config = ConfigDict(json_schema_extra={
        'example': {
            'message': 'Comment m'inscrire en master IADS ?',
            'session_id': 'sess_abc123',
        }
    })
```

```
message: str = Field(..., min_length=1, max_length=500)
session_id: str | None = None
```

[★] L'exemple apparait dans Swagger UI. L'utilisateur peut cliquer 'Use example' pour pre-remplir le body de test. Tres pratique pour la demo soutenance.

Chapitre 13 - Conclusion

13.1 Recap des concepts

- + FastAPI = framework Python async pour APIs REST
- + Routes = endpoints avec decorateurs `@router.get/post/...`
- + Pydantic = validation automatique des inputs
- + Async/await = non-bloquant pour de meilleures perfs
- + Depends() = injection de dependances
- + Routers = organisation modulaire des endpoints
- + Lifespan = init/cleanup au demarrage/arret
- + Swagger UI = documentation automatique interactive

13.2 Pour aller plus loin

- PDF 04 - pour comprendre comment FastAPI parle a MySQL via SQLAlchemy
- PDF 05 - pour MongoDB avec Motor (driver async)
- PDF 06 - pour le pipeline NLP/IA derriere le chatbot
- PDF 07 - pour la communication detaillee front/back et inter-services
- PDF 08 - pour la securite (JWT, bcrypt, etc.)

[★] Maitriser FastAPI te donne acces aux meilleures pratiques Python modernes. C'est un skill tres valorise sur le marche (Microsoft, Netflix, Uber l'utilisent).